Course: Parallel and Distributed Computing (ECOM 6339)

Student Name: Dalya Awad Mohammad Hassan

Student ID: 220253121

Assignment: Assignment 02

Date: 11 November 2026

Environment and Method

All experiments were run on a single Windows 11 PC using Ubuntu under WSL2 with OpenMPI (mpicc based on GCC 15.2) and a single-node Slurm 25.11 installation (slurmctld and slurmd configured by hand, partition 'main', node with 24 logical CPUs and about 31 GB RAM). Every program was submitted through sbatch; the job outputs are kept in the repository as *_<jobid>.out files.

Important caveat: all MPI processes ran on one WSL2 machine and communicated via shared memory, not an external network. Consequently, latency and bandwidth reflect the local MPI transport and memory system rather than network performance. The environment reported 24 logical CPUs; the physical-core configuration was not independently verified.

Task 1: Hello World

hello_mpi.c initialises MPI, queries the rank and size of MPI_COMM_WORLD, and prints "Hello from rank r of n".

```
MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);
printf("Hello from rank %d of %d\n", rank, size);
MPI_Finalize();
```

Compiled with mpicc -O2 and run with mpirun -np 4. It was then submitted with hello_mpi.slurm (#SBATCH --ntasks=4 --nodes=1), job 1. Captured output (hello_mpi_1.out):

```
Hello from rank 3 of 4
Hello from rank 1 of 4
Hello from rank 2 of 4
Hello from rank 0 of 4
```

The ranks print in arbitrary order because they execute independently and no synchronization orders their writes to standard output. All four ranks (0 to 3) appear exactly once.

Task 2: Ping-Pong Latency

pingpong.c exchanges a message between ranks 0 and 1 for 1 KB, 64 KB, 1 MB and 4 MB. For each size, 10 warm-up round trips are followed by 100 timed round trips measured with MPI_Wtime; the average round-trip time (RTT) is reported. Effective bandwidth is size / (RTT / 2). The experiment is done twice: with blocking MPI_Send/MPI_Recv and with MPI_Isend/MPI_Irecv completed by MPI_Wait (job 2, 2 ranks).

Size	Blocking RTT (us)	Blocking BW (MB/s)	Non-blocking RTT (us)	Non-blocking BW (MB/s)
1 KB	1.09	1886	3.30	622
64 KB	23.7	5528	35.7	3669
1 MB	163.9	12799	120.9	17353
4 MB	511.7	16394	425.2	19729

Table 1: Ping-pong results, average of 100 round trips (one run)

Discussion

- The 1 KB RTT of about 1.1 us (0.54 us one way) is the latency of a shared-memory transfer; it is far lower than a network latency would be.
- Bandwidth grows with message size because the fixed per-message cost is amortised, reaching about 16 to 20 GB/s for 4 MB, which is memory-copy speed.
- For small and medium messages (1 KB, 64 KB) the non-blocking version is slower: each call creates a request object and MPI_Wait must poll for progress, an overhead that matters when the transfer itself is short.
- For large messages (1 MB, 4 MB) the non-blocking version was about 20 to 30% faster in this run. A plausible reason is that the rendezvous handshake and the data copy are progressed together, but this explanation was not verified.
- In a ping-pong there is no independent computation to overlap with the communication, so non-blocking calls cannot show their real benefit (overlap) here; the differences are transport effects only. The results come from a single run on a non-dedicated machine and carry noise.

Task 3: Distributed Matrix Multiplication

matmul_mpi.c computes C = A x B for N = 2048 (double precision) with a row-block decomposition: each rank owns N/p consecutive rows of A and C. The algorithm is:

- Rank 0 generates A and B (fixed seed); MPI_Bcast sends the whole of B to all ranks.
- MPI_Scatter distributes N/p rows of A to each rank.
- Each rank computes its rows of C with an i-k-j loop nest (contiguous access to rows of B).
- MPI_Gather collects the rows of C on rank 0.
- Rank 0 recomputes C serially and compares it element by element (not included in timings).

Dalya Mohammad Hassan
220253121

```
int rows_per_rank = N / size;
MPI_Bcast(B, N*N, MPI_DOUBLE, 0, MPI_COMM_WORLD);
MPI_Scatter(A, rows_per_rank*N, MPI_DOUBLE, local_A, rows_per_rank*N, MPI_DOUBLE, 0,
MPI_COMM_WORLD);
/* local_C = local_A x B */
MPI_Gather(local_C, rows_per_rank*N, MPI_DOUBLE, C, rows_per_rank*N, MPI_DOUBLE, 0,
MPI_COMM_WORLD);
```

Correctness: for p = 1, 2, 4, 8 and 16 the maximum absolute difference to the serial reference was 0 (identical results, because each C element is accumulated in the same order), so every run is reported as PASS. Timing uses the maximum over ranks of each phase; 'comm' is the sum of the time spent in Bcast, Scatter and Gather.

p	Total (s)	Compute (s)	Comm (s)
1	3.516	3.459	0.057
2	2.318	2.266	0.052
4	1.548	1.479	0.094
8	1.179	1.114	0.097
16	1.133	1.002	0.191

Table 2: Single run (job 3), N = 2048

The final analysis in Section 5 uses the median of three repetitions (job 6) instead of this single run.

Task 4: Deadlock Demonstration

The deadlocking program

In deadlock.c (mode 'deadlock') both ranks first send 8 MB to the other rank and only then receive:

```
MPI_Ssend(sendbuf, COUNT, MPI_DOUBLE, other, 0, MPI_COMM_WORLD);
MPI_Recv (recvbuf, COUNT, MPI_DOUBLE, other, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

It was run through Slurm (job 4) under 'timeout 10 mpirun -np 2'. Both ranks printed their 'starting' line and then nothing more until the timeout killed the job (exit status 124):

```
=== 1. Original code (expected to HANG; killed after 10 s) ===
[rank 1] mode=deadlock: starting
[rank 0] mode=deadlock: starting
exit status: 124 (124 = killed by timeout, i.e. deadlock)
```

Why it deadlocks

A blocking send only returns when the send buffer can be reused. For large messages MPI uses a rendezvous protocol: the sender cannot complete until the receiver has posted a matching receive. Rank 0 sits in MPI_Send waiting for rank 1 to receive, and rank 1 sits in MPI_Send waiting for rank

0 to receive. Neither ever reaches its MPI_Recv, so there is a circular wait (rank 0 waits for rank 1, rank 1 waits for rank 0) and no progress is possible.

A plain MPI_Send with a small message often appears to work, because the implementation copies it into an internal buffer (eager protocol) and returns immediately. That is an implementation detail, not something the MPI standard guarantees, so such code is unsafe. To make the demonstration deterministic I used 8 MB messages and the synchronous MPI_Ssend, which never completes before the matching receive is posted.

Fixes

Fix	Idea	Result (job 4)
1. Reordering	Rank 0 sends then receives; rank 1 receives then sends. Breaks the symmetry so there is no cycle.	Completed, exit 0
2. MPI_Sendrecv	Send and receive are issued as one call; MPI schedules them to avoid the cycle.	Completed, exit 0
3. MPI_Isend / MPI_Irecv	Post both operations without blocking, then MPI_Waitall; both transfers proceed together.	Completed, exit 0

Table 3: Deadlock fixes

In all three fixed runs each rank received the other rank's data (rank 0 received value 1 from rank 1; rank 1 received value 0 from rank 0).

```
/* fix 1 */ if (rank==0){ Ssend(...); Recv(...);} else { Recv(...); Ssend(...);}
/* fix 2 */ MPI_Sendrecv(sendbuf,COUNT,MPI_DOUBLE,other,0, recvbuf,COUNT,MPI_DOUBLE,other,0,
...);
/* fix 3 */ MPI_Irecv(...,&reqs[0]); MPI_Isend(...,&reqs[1]); MPI_Waitall(2, reqs, ...);
```

Task 5: Scaling Analysis

Method and measurements

Task 3 was run three times for each p in {1, 2, 4, 8, 16} (job 6, all runs verified PASS) and the median is used. With T(1) = 3.892 s taken as the serial time:

- Speedup S(p) = T(1) / T(p)
- Efficiency E(p) = S(p) / p
- Total parallel overhead T_o(p) = p T(p) - T(1)
- Measured communication time = Bcast + Scatter + Gather (maximum over ranks)

p	T(p) s	Computes	Comms	Speedup	Efficiency	T_o = pT-T1 (s)	Comm / T(p)
1	3.892	3.846	0.033	1.00	1.00	0.00	0.8%
2	2.213	2.169	0.096	1.76	0.88	0.53	4.4%
4	1.610	1.536	0.085	2.42	0.60	2.55	5.3%
8	1.217	1.147	0.083	3.20	0.40	5.85	6.8%
16	1.290	1.176	0.141	3.02	0.19	16.74	10.9%

Table 4: Median of 3 runs, N = 2048

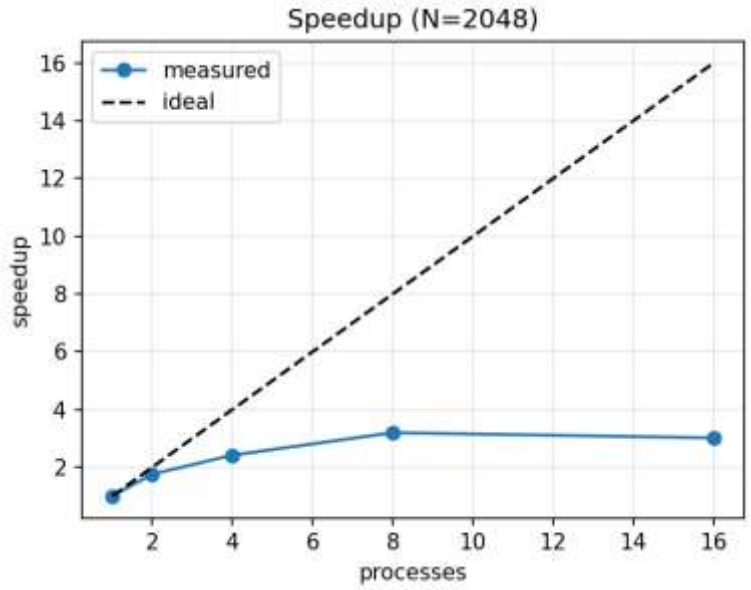


Figure 1: Speedup versus number of processes

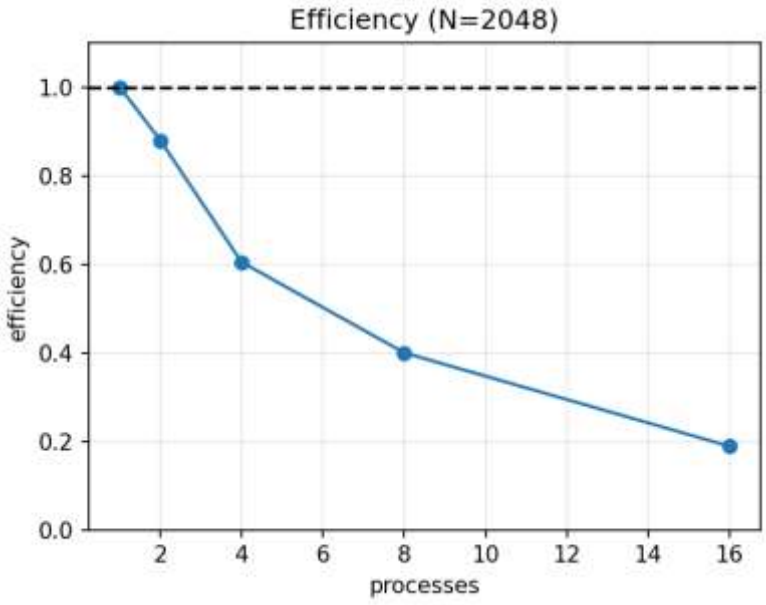


Figure 2: Parallel efficiency versus number of processes

Dalya Mohammad Hassan
220253121

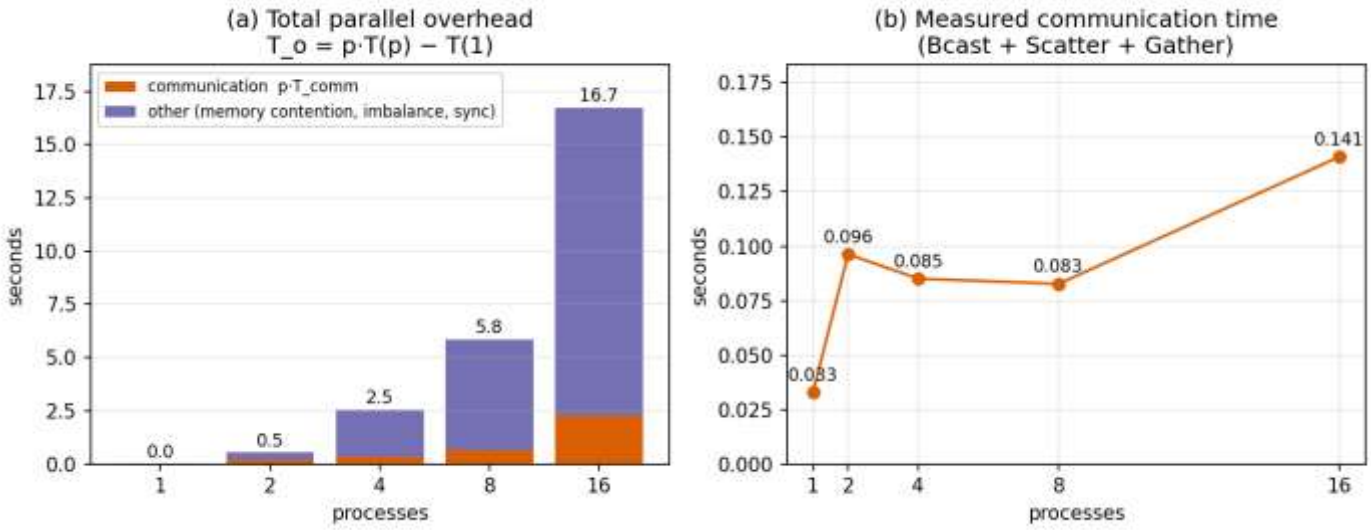


Figure 3: (a) Total parallel overhead, T_o(p) = pT(p) − T (1); (b) separately measured communication time for MPI_Bcast, MPI_Scatter, and MPI_Gather. These metrics use different scales and should not be interpreted as additive components.

Communication overhead

Communication time (MPI_Bcast, MPI_Scatter, and MPI_Gather) ranged from 0.083 to 0.141 seconds for p = 4–16 and measured 0.096 seconds at p = 2. It represented approximately 4.4% of total runtime at p = 2 and 10.9% at p = 16. Total parallel overhead, T_o(p) = pT(p) − T(1), increased from approximately 0.53 seconds at p = 2 to 16.74 seconds at p = 16. Figure 3 shows these metrics separately: the MPI communication measurements are phase-time estimates, whereas total overhead also reflects lost computational efficiency and other costs. The noncommunication contributions, such as memory contention, process scheduling, and imbalance, were not individually measured. Furthermore, the recorded computation and communication phase maxima can come from different ranks, so they should not be added to reconstruct total runtime.

Deviation from linear scaling

- Measured compute time decreased from 3.846 seconds at p = 1 to 1.147 seconds at p = 8, only about a 3.35-fold reduction rather than the ideal eightfold reduction. Each rank repeatedly accesses the complete 32 MB matrix B, which may constrain cache reuse and memory bandwidth. Hardware profiling would be required to confirm whether the kernel is predominantly memory bound.
- Increasing the process count from 8 to 16 did not improve median total runtime (1.217 seconds versus 1.290 seconds). The system reported 24 logical CPUs, but its physical-core topology was not verified. Memory contention, scheduling effects, and increasing communication costs may contribute to the observed loss of efficiency.

- Rank 0 owns the full input matrix A and output matrix C and handles the root-side communication in Scatter and Gather. Data generation and serial correctness verification were performed outside the timed interval.
- Scatter and Gather require root participation, while B is replicated on every process via MPI_Bcast. These communication steps may limit scaling, although their precise performance contributions depend on the MPI implementation.
- Single-machine measurements on a non-dedicated PC are noisy, which is why medians of three runs are used.

Isoefficiency

The serial work is W = Theta(N^3) (about 2 N^3 floating-point operations). The total parallel overhead is defined as T_o = p T(p) - T(1); communication is one contribution to it, not equal to it. To estimate how T_o grows, I assume a tree-based broadcast and that the communication of B dominates the overhead. Broadcasting B (N^2 elements) then costs about t_w N^2 log p + t_s log p per process, and scatter and gather add a further Theta(N^2) term. Summed over p processes:

```
T_o(p, N) = Theta( p * N^2 * log p )
```

For constant efficiency the work must grow in proportion to the overhead, W = Theta(T_o):

```
N^3 = Theta( p * N^2 * log p )   =>   N = Theta( p log p )   =>   W = N^3 = Theta( (p log p)^3 )
```

The estimated isoefficiency function is therefore W = Theta((p log p)^3). This is a theoretical estimate under the assumed tree-broadcast communication model, not an exact characterisation of every MPI implementation. It indicates that the problem size must grow rapidly with p to keep efficiency constant, so the algorithm scales poorly.

Memory is a second consideration: every rank stores the complete matrix B, which needs Theta(N^2) memory per rank and does not shrink as p grows. A two-dimensional decomposition such as SUMMA would reduce this replication and the per-process communication volume, to about Theta(N^2 / sqrt(p)).

The measurements show decreasing efficiency for a fixed-size problem: at N = 2048, efficiency fell from approximately 88% (p = 2) to 40% (p = 8) and 19% (p = 16). These observations are consistent with communication costs and possible memory-system or scheduling constraints, but the individual effects were not isolated. Figure 3 displays measured communication phase time separately from total parallel overhead. The asymptotic isoefficiency estimate has unspecified constants and is therefore not compared quantitatively with these measurements.

Conclusions

- MPI programs for all four source files compile and run under mpicc/mpirun and through Slurm.
- Shared-memory ping-pong gave about 1 us latency and up to about 20 GB/s bandwidth; non-blocking calls cost extra for small messages and helped for large ones.

Dalya Mohammad Hassan
220253121

- The row-block matrix multiplication is numerically correct against the serial reference and achieved approximately 3.20× speedup at eight processes. Scaling limitations may involve memory-system contention, scheduling, and MPI communication; their relative contributions were not independently established.
- The deadlock arises from a circular wait of two blocking synchronous sends and is removed by reordering, MPI_Sendrecv or non-blocking calls.
- Isoefficiency is Theta ((p log p)^3); a 2D decomposition would scale better.

Dalya Mohammad Hassan
220253121